1. University Scholarship Allocation

Problem statement

Evaluate applicants using aggregate marks, minimum marks in every subject, attendance, and family income. Rank only eligible applicants and apply clear tie-breaking rules.

Objectives

• Validate each applicant against all scholarship rules.

• Rank eligible applicants by aggregate marks, then attendance, then lower family income.

• Display eligible applicants in rank order and identify rejected applicants.

Assumptions

• Marks are percentages from 0 to 100; the example uses three subjects.

• Eligibility requires aggregate >= 80, every subject >= 60, attendance >= 75%, and annual family income <= 300000.

• Ties are resolved by higher attendance, then lower income; exact remaining ties keep input order.

Algorithm

1. Read applicant details and three subject marks.

2. Calculate aggregate as the average of subject marks.

3. Check every eligibility condition.

4. Store eligible records with aggregate, attendance, income, and name.

5. Sort using nested loops and swap when the next record has a better ranking.

6. Print ranks and a sample result.

Flowchart (text form)

```
START → Read applicants → Calculate aggregate → Check marks, attendance, income → Eligible? → Store eligible
```

Pseudocode

```
FOR each applicant\n  aggregate = average(subject marks)\n  IF aggregate >= 80 AND all subjects >= 60 AND attendance >= 75 AND in
```

1. University Scholarship Allocation — Python source code

Save the following as a .py file and run it with Python 3.

```
applicants = [
    ("Asha", [90, 85, 88], 92, 250000),
    ("Balu", [78, 84, 80], 88, 200000),
    ("Chitra", [82, 81, 85], 80, 280000),
    ("Deepak", [95, 58, 90], 95, 180000)
]
eligible = []
for name, marks, attendance, income in applicants:
    aggregate = sum(marks) / len(marks)
    if aggregate >= 80 and min(marks) >= 60 and attendance >= 75 and income <= 300000:
        eligible.append([name, aggregate, attendance, income])
    else:
        print(name, "NOT ELIGIBLE")
for i in range(len(eligible)):
    for j in range(i + 1, len(eligible)):
        a, b = eligible[i], eligible[j]
        if (b[1] > a[1] or
            (b[1] == a[1] and b[2] > a[2]) or
            (b[1] == a[1] and b[2] == a[2] and b[3] < a[3])):
            eligible[i], eligible[j] = eligible[j], eligible[i]
print("RANK  NAME    AGGREGATE  ATTENDANCE  INCOME")
for i, student in enumerate(eligible):
    print(i + 1, student[0], round(student[1], 2), student[2], student[3])
```

Test cases

Input / condition	Expected result
Asha: 90,85,88; attendance 92; income 250000	Eligible; aggregate 87.67; rank 1
Balu: 78,84,80; attendance 88; income 200000	Eligible; aggregate 80.67; rank 3
Deepak: 95,58,90; attendance 95; income 180000	Not eligible: one subject below 60

Sample output screenshot (console-style)

```
Deepak NOT ELIGIBLE\nRANK  NAME    AGGREGATE  ATTENDANCE  INCOME\n1 Asha 87.67 92 250000\n2 Chitra 82.67 80 2800
```

Conclusion

The program demonstrates multi-condition eligibility, list storage, and deterministic ranking. Real institutions should replace example thresholds with their official policy.

2. Progressive Electricity Billing

Problem statement

Calculate monthly household electricity bills using consumption slabs, fixed charges, surcharge, and rebate. Compare several months and flag unusual increases.

Objectives

• Apply progressive slab rates rather than one rate to all units.

• Add fixed charge and surcharge, then apply a rebate when eligible.

• Compare monthly bills and flag increases greater than 30%.

Assumptions

• Slabs: first 100 units at 2.00/unit, next 100 at 3.00/unit, above 200 at 5.00/unit.

• Fixed charge is 50; surcharge is 5% of energy charge; bills receive a 5% rebate if energy use is below 100 units.

• An increase over 30% from the previous month is unusual.

Algorithm

1. Read monthly unit readings.

2. Calculate energy charge progressively across three slabs.

3. Add fixed charge and surcharge.

4. Apply low-use rebate when applicable.

5. Compare each bill with the previous bill and report unusual increases.

Flowchart (text form)

```
START → Read monthly units → Calculate slab charge → Add fixed charge and surcharge → Apply rebate? → Store b
```

Pseudocode

```
FOR each month\n  charge = slab_cost(units)\n  bill = charge + fixed_charge + charge * surcharge_rate\n  IF units < 100: bill = b
```

2. Progressive Electricity Billing — Python source code

Save the following as a .py file and run it with Python 3.

```
months = [("Jan", 80), ("Feb", 110), ("Mar", 180), ("Apr", 260)]
bills = []
for month, units in months:
    if units <= 100:
        energy = units * 2
    elif units <= 200:
        energy = 100 * 2 + (units - 100) * 3
    else:
        energy = 100 * 2 + 100 * 3 + (units - 200) * 5
    bill = energy + 50 + energy * 0.05
    if units < 100:
        bill = bill - bill * 0.05
    bills.append(bill)
    print(month, units, round(bill, 2))
    if len(bills) > 1:
        increase = (bill - bills[-2]) / bills[-2] * 100
        if increase > 30:
            print("UNUSUAL INCREASE:", round(increase, 1), "%")
```

Test cases

Input / condition	Expected result
80 units	80 units: energy 160; subtotal including fixed and surcharge 218; 5% rebate gives 207.10
260 units	Energy 600; fixed 50; surcharge 30; bill 680
Month-to-month rise > 30%	Prints UNUSUAL INCREASE

Sample output screenshot (console-style)

```
Jan 80 207.1
Feb 110 291.5
UNUSUAL INCREASE: 40.8 %
Mar 180 512.0
UNUSUAL INCREASE: 75.6 %
Apr 260 680.0
UNUSUAL INCREASE: 32.8 %
```

Conclusion

Progressive slabs and month-to-month comparisons make billing transparent. The example rebate is applied to the full subtotal for units below 100.

3. Hospital Billing and Patient Classification

Problem statement

Prepare a patient bill from consultation, diagnostic, and treatment charges; apply eligible concessions, validate the bill, and classify payment status using a policy.

Objectives

• Calculate the total charge and concession.

• Prevent negative charges and concession values outside the allowed range.

• Classify accounts as PAID, PARTIAL, or DUE.

Assumptions

• All charges must be non-negative.

• Senior patients receive a 10% concession on charges before tax; other patients receive no concession.

• Payment policy: paid >= bill means PAID; paid > 0 means PARTIAL; otherwise DUE. No tax is applied in this example.

Algorithm

1. Read patient details and charges.

2. Validate charges and payment.

3. Calculate subtotal and any concession.

4. Calculate final bill and outstanding amount.

5. Classify payment and print a statement.

Flowchart (text form)

```
START → Read charges/payment → Validate values → Calculate subtotal → Apply concession → Compare payment with
```

Pseudocode

```
subtotal = consultation + diagnostics + treatment\nIF senior: concession = subtotal * 10% ELSE concession = 0\nbill = subtotal -
```

3. Hospital Billing and Patient Classification — Python source code

Save the following as a .py file and run it with Python 3.

```
patient = "Ravi"
senior = True
consultation, diagnostics, treatment = 500, 1200, 3000
payment = 2500
if min(consultation, diagnostics, treatment, payment) < 0:
    print("INVALID BILL: negative amount")
else:
    subtotal = consultation + diagnostics + treatment
    if senior:
        concession = subtotal * 0.10
    else:
        concession = 0
    bill = subtotal - concession
    balance = bill - payment
    if payment >= bill:
        status = "PAID"
    elif payment > 0:
        status = "PARTIAL"
    else:
        status = "DUE"
    print("Patient:", patient)
    print("Subtotal:", subtotal)
    print("Concession:", round(concession, 2))
    print("Final bill:", round(bill, 2))
    print("Balance:", round(max(0, balance), 2))
    print("Status:", status)
```

Test cases

Input / condition	Expected result
Senior; charges 500,1200,3000; payment 2500	Subtotal 4700; concession 470; bill 4230; balance 1730; PARTIAL
Any charge = -10	INVALID BILL
Payment >= final bill	Status PAID

Sample output screenshot (console-style)

```
Patient: Ravi\nSubtotal: 4700\nConcession: 470.0\nFinal bill: 4230.0\nBalance: 1730.0\nStatus: PARTIAL
```

Conclusion

This case uses conditionals to validate input and classify payment. Hospital-specific concessions and payment rules must be supplied by the institution.

4. Employee Payroll and Deduction Analysis

Problem statement

Calculate gross salary from basic pay, allowances, overtime, and incentives; apply progressive deductions and print a formatted salary statement.

Objectives

• Calculate gross salary accurately.

• Apply increasing deduction rates to income bands.

• Display gross pay, deductions, and net pay.

Assumptions

• Allowances are entered as amounts; overtime pay is hours multiplied by hourly overtime rate.

• Deduction: 0% on first 15000, 5% on the next 10000, and 10% on gross salary above 25000.

• All amounts are non-negative; this is an illustrative policy, not tax advice.

Algorithm

1. Read salary components.

2. Calculate overtime and gross salary.

3. Calculate deduction for each progressive band.

4. Subtract deductions from gross salary.

5. Print a salary statement.

Flowchart (text form)

```
START → Read salary data → Validate → Calculate overtime → Calculate gross → Calculate progressive deduction
```

Pseudocode

```
overtime_pay = hours * overtime_rate\ngross = basic + allowances + overtime_pay + incentive\ndeduction = 0\nApply 0% to first 150
```

4. Employee Payroll and Deduction Analysis — Python source code

Save the following as a .py file and run it with Python 3.

```
name = "Meena"
basic, allowances = 22000, 5000
hours, overtime_rate, incentive = 5, 200, 1000
overtime = hours * overtime_rate
gross = basic + allowances + overtime + incentive
if min(basic, allowances, hours, overtime_rate, incentive) < 0:
    print("INVALID INPUT")
else:
    deduction = 0
    if gross > 15000:
        deduction += min(gross - 15000, 10000) * 0.05
    if gross > 25000:
        deduction += (gross - 25000) * 0.10
    net = gross - deduction
    print("SALARY STATEMENT")
    print("Employee:", name)
    print("Basic:", basic, "Allowances:", allowances)
    print("Overtime:", overtime, "Incentive:", incentive)
    print("Gross:", round(gross, 2))
    print("Deduction:", round(deduction, 2))
    print("Net salary:", round(net, 2))
```

Test cases

Input / condition	Expected result
Basic 22000; allowances 5000; overtime 1000; incentive 1000	Gross 29000; deduction 900; net 28100
Negative salary component	INVALID INPUT
Gross 12000	Deduction 0; net 12000

Sample output screenshot (console-style)

```
SALARY STATEMENT
Employee: Meena
Basic: 22000 Allowances: 5000
Overtime: 1000 Incentive: 1000
Gross: 29000
Deduction: 900.0
Net salary: 28100.0
```

Conclusion

The salary statement shows how variables, arithmetic, conditionals, and formatted output can support payroll calculations.

5. Multi-Plan Telecommunications Cost Optimization

Problem statement

Compare mobile plans against usage, validity, and budget limits. Calculate excess charges and select the cheapest plan that satisfies all constraints.

Objectives

• Calculate each plan's total cost for the supplied usage.

• Reject plans with insufficient validity or budget excess.

• Select the most economical eligible plan.

Assumptions

• Usage is measured in GB; included data is consumed first.

• Plans are eligible only when validity is at least required days and total cost is within budget.

• Excess data is charged at each plan's listed per-GB rate.

Algorithm

1. Read usage, required validity, and budget.

2. For each plan, calculate excess data and total cost.

3. Check validity and budget.

4. Keep the eligible plan with the lowest cost.

5. Print all plan costs and the best plan.

Flowchart (text form)

```
START → Read usage/budget → Evaluate each plan → Calculate excess charges → Check validity and budget → Eligi
```

Pseudocode

```
best_plan = NONE\nFOR each plan\n  excess = max(0, usage - included_data)\n  cost = base_price + excess * excess_rate\n  IF valid
```

5. Multi-Plan Telecommunications Cost Optimization — Python source code

Save the following as a .py file and run it with Python 3.

```
usage, required_days, budget = 12, 28, 400
plans = [
    ("Basic", 199, 10, 28, 20),
    ("Smart", 299, 15, 28, 10),
    ("Flex", 349, 20, 56, 5)
]
best_name, best_cost = "NONE", 0
for name, price, data, days, rate in plans:
    excess = max(0, usage - data)
    cost = price + excess * rate
    print(name, "cost =", cost)
    if days >= required_days and cost <= budget:
        if best_name == "NONE" or cost < best_cost:
            best_name, best_cost = name, cost
if best_name == "NONE":
    print("No eligible plan")
else:
    print("Best plan:", best_name, "Total:", best_cost)
```

Test cases

Input / condition	Expected result
Usage 12 GB; 28 days; budget 400	Basic 239; Smart 299; Flex 349; best Basic
Budget 200	No plan eligible because Basic costs 239
Required validity 60 days	No plan eligible

Sample output screenshot (console-style)

```
Basic cost = 239\nSmart cost = 299\nFlex cost = 349\nBest plan: Basic Total: 239
```

Conclusion

The comparison avoids choosing a plan by headline price alone. The cheapest valid plan is selected only after all constraints are checked.

6. Railway Fare and Reservation Calculator

Problem statement

Calculate fares for multiple passengers using distance slabs, travel class, age concessions, reservation fees, and cancellation penalties.

Objectives

• Compute distance-based base fare and class multiplier.

• Apply child and senior concessions according to the sample policy.

• Calculate reservation fees and cancellation deductions.

Assumptions

• Base fare: first 100 km at 1.00/km, next 200 km at 0.80/km, remaining distance at 0.60/km.

• Class multipliers: Sleeper 1.0, AC 2.0.

• Children under 12 receive 50% fare concession; passengers aged 60+ receive 30%. Reservation fee is 40 per passenger. Cancellation penalty is 25% of the fare before reservation fee.

Algorithm

1. Read passenger name, age, distance, class, and cancellation choice.

2. Calculate distance-slab fare and class multiplier.

3. Apply age concession and add reservation fee.

4. If cancelled, calculate penalty and refund; otherwise show payable fare.

5. Repeat for each passenger.

Flowchart (text form)

```
START → Read passenger → Validate distance/age → Calculate distance fare → Apply class multiplier → Apply age
```

Pseudocode

```
Calculate slab fare from distance\nMultiply by class factor\nIF age < 12: fare = fare * 0.5\nELSE IF age >= 60: fare = fare * 0.
```

6. Railway Fare and Reservation Calculator — Python source code

Save the following as a .py file and run it with Python 3.

```
passengers = [
    ("Anu", 10, 250, "Sleeper", False),
    ("Ramesh", 65, 350, "AC", True)
]
for name, age, km, travel_class, cancelled in passengers:
    if km <= 100:
        base = km * 1.0
    elif km <= 300:
        base = 100 + (km - 100) * 0.8
    else:
        base = 100 + 200 * 0.8 + (km - 300) * 0.6
    if travel_class == "AC":
        base = base * 2
    if age < 12:
        base = base * 0.5
    elif age >= 60:
        base = base * 0.7
    total = base + 40
    if cancelled:
        penalty = base * 0.25
        print(name, "CANCELLED", "Penalty:", round(penalty, 2),
              "Refund:", round(total - penalty, 2))
    else:
        print(name, "PAYABLE:", round(total, 2))
```

Test cases

Input / condition	Expected result
Anu; age 10; 250 km; Sleeper; not cancelled	Base 220; concession fare 110; total 150
Ramesh; age 65; 350 km; AC; cancelled	Distance fare 290; AC fare 580; senior fare 406; penalty 101.5; total 446; refund 344.5
Distance = 0	Should be rejected by a production version; add km <= 0 validation

Sample output screenshot (console-style)

```
Anu PAYABLE: 150.0\nRamesh CANCELLED Penalty: 101.5 Refund: 344.5
```

Conclusion

The program combines loops, tuples, conditions, and slab calculations. Fare rules are illustrative and should be replaced with official railway rules.

7. Retail Billing with Multiple Discount Rules

Problem statement

Prepare an itemized retail invoice with product quantities, category-specific tax, threshold discounts, and a maximum discount limit.

Objectives

• Calculate line totals and category taxes.

• Apply a bill-level discount based on subtotal thresholds.

• Limit the discount to a stated maximum and print the invoice.

Assumptions

• Tax: food 5%, clothing 12%, electronics 18%.

• Subtotal discount is 10% when subtotal >= 5000, otherwise 5% when subtotal >= 2000; maximum discount is 1000.

• Tax is calculated on the subtotal after discount, using each item's category rate.

Algorithm

1. Read products as tuples containing name, category, unit price, and quantity.

2. Calculate each line amount and subtotal.

3. Select the discount rate and apply maximum cap.

4. Allocate discount proportionally to each line, calculate category tax, and print totals.

Flowchart (text form)

```
START → Read products → Calculate line totals/subtotal → Choose discount rate → Apply discount cap → Calculat
```

Pseudocode

```
subtotal = sum(price * quantity)\nChoose discount rate based on subtotal\ndiscount = min(subtotal * rate, 1000)\nFOR each produc
```

7. Retail Billing with Multiple Discount Rules — Python source code

Save the following as a .py file and run it with Python 3.

```
products = [
    ("Rice", "food", 80, 10),
    ("Shirt", "clothing", 700, 3),
    ("Headphones", "electronics", 1200, 2)
]
rates = {"food": 0.05, "clothing": 0.12, "electronics": 0.18}
subtotal = 0
for name, category, price, qty in products:
    subtotal += price * qty
if subtotal >= 5000:
    rate = 0.10
elif subtotal >= 2000:
    rate = 0.05
else:
    rate = 0
discount = min(subtotal * rate, 1000)
tax_total = 0
print("ITEM       AMOUNT   TAX")
for name, category, price, qty in products:
    line = price * qty
    share = discount * line / subtotal
    taxable = line - share
    tax = taxable * rates[category]
    tax_total += tax
    print(name, line, round(tax, 2))
print("Subtotal:", subtotal)
print("Discount:", round(discount, 2))
print("Tax:", round(tax_total, 2))
print("Grand total:", round(subtotal - discount + tax_total, 2))
```

Test cases

Input / condition	Expected result
Products shown in code	Subtotal 5600; discount 560; tax calculated by category
Subtotal 1500	No discount
Subtotal 20000	Discount capped at 1000

Sample output screenshot (console-style)

```
ITEM       AMOUNT   TAX
Rice 800 38.0
Shirt 2100 226.8
Headphones 2400 388.8
Subtotal: 5300
Discount: 530.0
Tax: 653.6
Grand total: 5423.6
```

Conclusion

This invoice demonstrates tuple unpacking, lists, category-based conditions, loops, and proportional discount allocation.

8. Manufacturing Cost and Profitability Analysis

Problem statement

Calculate manufacturing cost from materials, labour, wastage, and machine costs, then decide whether the selling price meets a profit target.

Objectives

• Include wastage and all production costs.

• Calculate unit cost, revenue, profit, and profit percentage.

• Check whether the required profit target is achieved.

Assumptions

• Material cost is increased by a wastage percentage.

• Labour and machine costs are totals for the batch.

• Profit target is 20% of total production cost; selling price is per finished unit.

Algorithm

1. Read batch quantity and cost values.

2. Calculate material cost including wastage.

3. Add labour and machine costs.

4. Calculate revenue, profit, and profit percentage.

5. Compare profit percentage with the target.

Flowchart (text form)

```
START → Read batch and cost values → Add material wastage → Calculate total cost → Calculate revenue/profit →
```

Pseudocode

```
material_total = material_cost * quantity\nwastage_cost = material_total * wastage_rate\ntotal_cost = material_total + wastage_co
```

8. Manufacturing Cost and Profitability Analysis — Python source code

Save the following as a .py file and run it with Python 3.

```
quantity = 100
material_per_unit = 40
wastage_rate = 0.08
labour, machine = 1200, 800
selling_price = 65
target = 20
material = quantity * material_per_unit
wastage = material * wastage_rate
total_cost = material + wastage + labour + machine
revenue = quantity * selling_price
profit = revenue - total_cost
profit_percent = profit / total_cost * 100
print("Material:", material)
print("Wastage:", round(wastage, 2))
print("Total cost:", round(total_cost, 2))
print("Revenue:", revenue)
print("Profit:", round(profit, 2))
print("Profit %:", round(profit_percent, 2))
if profit_percent >= target:
    print("TARGET MET")
else:
    print("TARGET NOT MET")
```

Test cases

Input / condition	Expected result
100 units; material 40/unit; 8% wastage; labour 1200; machine 800; price 65	Total cost 6320; revenue 6500; profit 180; 2.85%; target not met
Selling price 80	Higher profit; target met
Quantity <= 0	A production version should reject invalid quantity

Sample output screenshot (console-style)

```
Material: 4000\nWastage: 320.0\nTotal cost: 6320.0\nRevenue: 6500\nProfit: 180.0\nProfit %: 2.85\nTARGET NOT MET
```

Conclusion

The model helps identify whether a production batch earns the required margin and highlights the importance of including wastage and machine costs.

9. Multi-Currency Travel Budget Analysis

Problem statement

Convert travel expenses from foreign currencies using supplied exchange rates, include conversion fees, compare total spending with a budget, and find the largest category.

Objectives

• Convert expenses to the home currency.

• Add a conversion fee to each converted amount.

• Compare spending with budget and identify the largest category.

Assumptions

• Exchange rates are supplied as home-currency units per one foreign-currency unit.

• A 2% conversion fee applies to each expense.

• All example expenses are positive; budget is 100000 home-currency units.

Algorithm

1. Read expense category, foreign amount, and exchange rate.

2. Convert amount and add fee.

3. Accumulate total and track the largest expense.

4. Compare total with budget and print results.

Flowchart (text form)

```
START → Read expense records and rates → Convert currency → Add 2% fee → Add to total → Track largest categor
```

Pseudocode

```
total = 0; largest_amount = 0\nFOR each expense tuple\n  converted = foreign_amount * exchange_rate\n  final_amount = converted +
```

9. Multi-Currency Travel Budget Analysis — Python source code

Save the following as a .py file and run it with Python 3.

```
expenses = [
    ("Hotel", 300, 83.0),
    ("Food", 120, 83.0),
    ("Transport", 80, 90.0),
    ("Shopping", 150, 83.0)
]
total, largest, largest_name = 0, 0, ""
for category, amount, rate in expenses:
    converted = amount * rate
    final_amount = converted * 1.02
    total += final_amount
    print(category, "=", round(final_amount, 2))
    if final_amount > largest:
        largest, largest_name = final_amount, category
budget = 100000
print("Total:", round(total, 2))
print("Largest category:", largest_name)
if total <= budget:
    print("WITHIN BUDGET; remaining:", round(budget - total, 2))
else:
    print("OVER BUDGET by:", round(total - budget, 2))
```

Test cases

Input / condition	Expected result
Expenses listed in code	Total 55600.2; largest category Hotel; within budget
Budget 5000	Over budget by 666.7
Change hotel amount to 500	Hotel remains largest category

Sample output screenshot (console-style)

```
Hotel = 25398.0\nFood = 10159.2\nTransport = 7344.0\nShopping = 12699.0\nTotal: 55600.2\nLargest category: Hotel
```

Conclusion

The case demonstrates tuple records, exchange-rate multiplication, fee calculations, totals, and maximum tracking.

10. Campus Event Financial Planning

Problem statement

Calculate event costs for venue, equipment, catering, transport, and certificates. Determine participant contributions, sponsorship shortfall, and minimum registration count to break even.

Objectives

• Calculate fixed and variable costs.

• Estimate the amount that sponsorship must cover.

• Find the minimum registrations needed when each registration contributes a fixed fee.

Assumptions

• Venue, equipment, and transport are fixed costs; catering and certificates are per participant.

• Sponsorship is subtracted from total cost.

• Registration fee is 500 per participant; minimum count is the smallest integer n for which registration income plus sponsorship covers costs.

Algorithm

1. Read fixed costs, per-person costs, sponsorship, and fee.

2. Calculate cost at the expected attendance.

3. Calculate funding gap after sponsorship.

4. Increase participant count from 0 until income plus sponsorship covers fixed and variable costs.

5. Print budget and break-even count.

Flowchart (text form)

```
START → Read costs and fee → Calculate event cost → Subtract sponsorship → Calculate shortfall → Test partici
```

Pseudocode

```
fixed = venue + equipment + transport\nvariable_per_person = catering + certificate\ncost = fixed + expected_people * variable_pe
```

10. Campus Event Financial Planning — Python source code

Save the following as a .py file and run it with Python 3.

```
venue, equipment, transport = 12000, 5000, 3000
catering_per_person, certificate_per_person = 180, 20
sponsorship, fee, expected = 10000, 500, 80
fixed = venue + equipment + transport
variable = catering_per_person + certificate_per_person
total_cost = fixed + expected * variable
income = sponsorship + expected * fee
shortfall = max(0, total_cost - income)
n = 0
while sponsorship + n * fee < fixed + n * variable:
    n += 1
print("Fixed costs:", fixed)
print("Expected total cost:", total_cost)
print("Sponsorship:", sponsorship)
print("Registration income:", expected * fee)
print("Funding shortfall:", shortfall)
print("Break-even registrations:", n)
```

Test cases

Input / condition	Expected result
80 expected participants	Total cost 36000; sponsorship 10000; registration income 40000; shortfall 0; break-even count 34
No sponsorship	Break-even count 50 because fixed 20000 and net contribution 300 per person
Fee <= variable cost	If sponsorship is less than fixed costs, break-even may not be reachable; validate policy

Sample output screenshot (console-style)

```
Fixed costs: 20000
Expected total cost: 36000
Sponsorship: 10000
Registration income: 40000
Funding shortfall: 0
Break-even registrations: 34
```

Conclusion

The planner separates fixed and per-participant costs, checks funding, and uses a loop to find the minimum break-even attendance.